

FinePrint — Complete Project Analysis

An AI-Powered Contract Risk Monitoring & Governance Platform

A deep, beginner-friendly, interview-ready breakdown

Table of Contents

- 1. [What FinePrint Does & Why It Exists](#)
- 2. [Simple Overview → Deep Technical Overview](#)
- 3. [End-to-End Workflow](#)
- 4. [Codebase Map — Every Major Component](#)
- 5. [Every Technology Explained \(What / Why / Where / How\)](#)
- 6. [AI Deep Dive — LangGraph, LangChain, RAG, Agents, Embeddings](#)
- 7. [How Everything Connects](#)
- 8. [Architecture Diagrams](#)
- 9. [Complete Data Flow: Input → Output](#)
- 10. [Walkthrough Example \(Realistic, Step-by-Step\)](#)
- 11. [File-by-File / Function-by-Function Reference](#)
- 12. [Database Schema Explained](#)
- 13. [FAANG-Level Interview Preparation](#)
- 14. [Design Decisions & Trade-offs](#)
- 15. [Quick-Reference Cheat Sheet](#)

1. What FinePrint Does & Why It Exists

The plain-English version

Imagine your company signs 40 different vendor contracts a year — software subscriptions, marketing agencies, cloud hosting, freelancers. Buried inside the legal text of each one are traps:

- **Auto-renewal clauses** that silently lock you in for another year unless you cancel 90 days early.
- **Termination penalties** — pay 30% extra if you leave early.
- **Price escalation clauses** — your bill goes up 5-10% every year automatically.
- **Dead software** — you're paying for 15 licenses but only 2 people still log in.

No human legal team has time to re-read every contract every month and cross-reference it with "who's actually still using this." **FinePrint is a robot paralegal + financial analyst that never sleeps.** It:

1. **Reads** every contract (PDF/DOCX/TXT).
2. **Understands** the risky clauses using an LLM (AI).
3. **Calculates** the exact dollar exposure using plain math (not AI — this part is 100% deterministic).
4. **Decides** what to do (cancel / renegotiate / renew) and explains *why*.
5. **Checks itself** against company policy — if the money at stake is large, it refuses to act alone and asks a human.
6. **Drafts** the email/Slack message for that human to review.
7. **Sends** it only after a person clicks "Approve" and "Send."
8. **Comes back 30 days later** to verify whether it actually worked (did the vendor really get cancelled?).

The problem it solves

Without FinePrint	With FinePrint
Contracts silently auto-renew, nobody notices until the invoice arrives	System flags renewals weeks in advance
Legal/finance teams manually skim every contract	AI extracts structured facts automatically
No visibility into "are we actually using what we pay for?"	Cross-references usage/seat data

Without FinePrint	With FinePrint
Risky, high-dollar decisions could be made by an AI with no oversight	Deterministic policy engine forces human approval above a \$ threshold
No record of <i>why</i> a decision was made	Full audit trail: AI reasoning + policy check + human action, all logged

The one big idea behind the whole project

"AI suggests. Rules govern. Humans approve."

This is the single most important concept in FinePrint, and it's the answer to almost every "why did you build it this way?" interview question. The LLM is trusted to *read and summarize* (something it's good at), but it is **never trusted to do arithmetic or make the final call on real money** — that's hard-coded Python logic and a human being.

2. Simple Overview → Deep Technical Overview

2.1 Simple overview (no jargon)

FinePrint has two halves that talk to each other over the network:

- **A website** (the "Dashboard") — built with Next.js/React — where a person uploads contracts, watches risk scores, and clicks Approve/Reject.
- **A brain** (the "Backend") — built with FastAPI (Python) — that does the actual reading, thinking, and deciding, and stores everything in a database.

Inside the brain, there's a **pipeline** — think of it like a factory assembly line. A contract enters the pipeline as raw text and exits as a "recommendation" with a risk level and a dollar figure attached. This assembly line has five stations (called "agents"):



2.2 Deep technical overview

Architecturally, FinePrint is a **modular monolith** — a single FastAPI application (not microservices) with clearly separated internal modules. Its core "thinking" component is a **LangGraph** `StateGraph` — a finite-state machine where each node is a Python function that receives a shared `TypedDict` state object, does work, and returns an updated version of that state. The graph is checkpointable (state persists between steps), conditionally branches (e.g., low-confidence extraction → manual review), and is invoked as a background task per contract scan.

Four of the five pipeline nodes call an LLM (via the **Groq** API, running an open-weight model such as Qwen) using carefully constrained prompts that force raw JSON output. The **Finance node deliberately makes zero LLM calls** — it's pure Python arithmetic — because letting a language model compute money is a design anti-pattern the team explicitly avoided (LLMs can "hallucinate" numbers).

After the LLM proposes a recommendation, a **separate, non-AI deterministic rule engine** (`app/rules/decision_rules.py`) compares the proposed dollar impact against an org-configurable threshold stored in Postgres. If it exceeds the threshold (or risk = high), the system sets `requires_approval = True` and the graph terminates at a `notify` node — it does **not** proceed to action automatically.

All external system access (Slack, Google Drive, Gmail, Okta/usage-data) is abstracted behind the **Model Context Protocol (MCP)** — a standard client/server protocol for exposing "tools" (functions) to AI systems. FinePrint never hard-codes a Slack SDK call directly into

business logic; instead it asks an MCP client for a `post_message` tool and calls it, so swapping/adding integrations doesn't require touching the agent code.

For long-term memory, FinePrint stores an **embedding vector** (`pgvector`, 1536-dim) on every `Decision` row and does a similarity search (`12_distance`) to retrieve the 3 most similar **past decisions with verified successful outcomes**, injecting them into the Decision Agent's prompt as historical grounding — a lightweight, in-house version of **Retrieval-Augmented Generation (RAG)**.

3. End-to-End Workflow

Here is the full lifecycle of one contract, beginning to end:



STAGE 5 – VERIFICATION

Verifier job re-queries usage data via MCP.

Active users == 0 → outcome = "success", records real \$ saved.

Otherwise → outcome = "failure"/"inconclusive".

This outcome later feeds the RAG context for future Decision Agent runs.

4. Codebase Map – Every Major Component

FinePrint/

└─ backend/	← FastAPI Python backend (the "brain")
└─ app/	
└─ main.py	– FastAPI app entryptoint, starts background schedulers
└─ config.py	– Central settings (env vars via pydantic-settings)
└─ database.py	– SQLAlchemy engine/session setup
└─ orchestrator/	– THE BRAIN'S BRAIN (LangGraph)
└─ state.py	– ContractScanState (shared data passed node→node)
└─ graph.py	– Builds & compiles the LangGraph state machine
└─ agents/	– The 5 pipeline "workers"
└─ detection.py	– LLM: raw text → structured clauses (JSON)
└─ risk.py	– LLM: clauses + usage → risk classification
└─ finance.py	– Pure Python: exact dollar exposure (NO LLM)
└─ decision.py	– LLM + RAG: final recommendation
└─ action.py	– LLM: drafts the outbound email/Slack message
└─ rules/	
└─ decision_rules.py	– Deterministic approval-threshold logic (NO LLM)
└─ mcp/	– Model Context Protocol integration layer
└─ client_manager.py	– Manages MCP sessions per org (Mock + Real)
└─ crypto.py	– Encrypts/decrypts stored MCP credentials
└─ drive_tools.py / gmail_tools.py / slack_tools.py / okta_tools.py	
└─ ingestion/	– Pulls contracts in
└─ ingestion_service.py	– Polls Drive/Gmail MCP tools
└─ document_parser.py	– PDF/DOCX/TXT → plain text
└─ deduplication.py	– SHA-256 hash dedup
└─ scheduler.py	– Background polling loop (thread)
└─ verification/	
└─ verifier.py	– 30-day outcome-checking background job
└─ jobs/	
└─ retention.py	– Data retention/cleanup job
└─ api/	– REST endpoints (FastAPI routers)
└─ auth.py	– register/login/me (JWT)
└─ contracts.py	– upload/list/detail/trigger-scan
└─ decisions.py	– approval queue, approve/reject
└─ actions.py	– draft view, send action
└─ dashboard.py	– summary stats for the UI
└─ mcp.py	– connect/disconnect MCP integrations
└─ settings.py	– org policy thresholds
└─ audit.py	– immutable audit trail
└─ auth/	
└─ jwt.py	– token creation/validation
└─ rbac.py	– role-based access control (admin/user/viewer)

services/	
email_service.py	- SMTP notification emails
slack_service.py	- Slack notifications via MCP
models/	- SQLAlchemy ORM tables (see §12)
alembic/	- Database migrations (version-controlled schema changes)
langgraph.json	- LangGraph CLI/dev-server config
requirements.txt	- Python dependencies
frontend/	← Next.js React dashboard (the "face")
src/	
app/	- Pages: dashboard, contracts, approvals, actions, activity, settings, login (Next.js App Router)
components/	- RiskBadge, RiskMeter, AgentTrail, PipelineNode, StatCard, CommandPalette, Sidebar, etc.
lib/api.ts	- typed fetch wrapper calling the FastAPI backend
DOC/	- Design documents (SRS, HLD/LLD, System Design)
test_docs/	- Sample contracts used for demos/testing
mock_drive_mcp.py / mock_slack_mcp.py / sqlite_mcp.py	- standalone MCP servers
docker-compose.yml	- Local orchestration (Postgres, Redis, backend, frontend)

5. Every Technology Explained (What / Why / Where / How)

Technology	What it is	Why FinePrint uses it	Where it's used	How it works here
FastAPI	Python web framework for building APIs, built on Starlette + Pydantic	Async-friendly, auto-generates OpenAPI docs (/docs), type-safe request/response models, fast to build with	backend/app/main.py, all of app/api/*	Routers (APIRouter) are domain (contracts, decisions) and mounted with prefix. Depends() injects DB session authenticated user into request.
SQLAlchemy 2.0 (ORM)	Python library that maps database tables to Python classes	Lets the team write Python objects (Contract, Decision) instead of raw SQL, with relationships (contract.decisions)	app/models/*.py, app/database.py	Uses the modern typed mapped_column() style. Event inherits UUIDPrimaryKeyMixin and TimestampMixin for consistency created_at, updated_at.
Alembic	Database migration tool, sister project to SQLAlchemy	Tracks schema changes over time (e.g., "add embedding column to decisions") as versioned, reversible scripts	backend/alembic/versions/*.py	Each migration file has upgrade() / downgrade() - d7749d464f11_add_embedding added the pgvector column to project's life.
PostgreSQL + pgvector	Relational database; pgvector is an extension adding a native VECTOR column type + similarity search	Postgres for structured, relational, audit-friendly data; pgvector to store LLM embeddings and run similarity search for the RAG-like historical-context feature	decisions.embedding column (Vector(1536))	Decision.embedding.l2_distance in SQLAlchemy is translated to Postgres vector-distance ascending (closest first).
Pydantic / pydantic-settings	Data-validation library; pydantic-settings loads typed config from .env files	Strong typing + validation for API request/response bodies and app configuration	app/config.py (Settings(BaseSettings)), request models throughout api/	Settings reads .env, cache approval_threshold_usd: 10000 wrapped in @lru_cache() once as a singleton.

Technology	What it is	Why FinePrint uses it	Where it's used	How it works here
Groq (LLM inference API)	A very fast LLM-inference cloud provider (serves open-weight models like Qwen/Llama at high tokens/sec)	Speed — contract scanning needs multiple sequential LLM calls per document; Groq's low latency keeps the pipeline fast and cheap	<code>agents/detection.py</code> , <code>risk.py</code> , <code>decision.py</code> , <code>action.py</code>	Each agent creates a <code>Gr</code> calls <code>chat.completions.create(messages=[...], temperature=0.0)</code> is used for extraction/classification (deterministic-ish output more "creative" email drafts)
LangGraph	A graph/state-machine orchestration library (from the LangChain ecosystem) for building multi-step, stateful, conditionally-branching LLM pipelines	Gives FinePrint a structured, debuggable, checkpointable pipeline instead of one giant prompt or a tangle of if/else calling functions	<code>app/orchestrator/graph.py</code> , <code>state.py</code>	<code>StateGraph(ContractScanS</code> are added with <code>add_node</code> <code>add_edge</code> (unconditional), <code>add_conditional_edges</code> (based on a router function's state value). Compiled once into <code>fineprint_graph</code> .
LangChain / langchain-groq	Framework providing common LLM abstractions (chat models, prompt templates, tool-calling)	Present in <code>requirements.txt</code> as the ecosystem; LangGraph is built on top of / integrates with (<code>langchain-groq</code> provides a LangChain-compatible wrapper for Groq models)	<code>requirements.txt</code> ; available for future use (e.g., swapping the raw <code>groq</code> SDK calls for <code>ChatGroq</code>)	In the current code, agents use <code>groq</code> Python SDK directly going through <code>langchain</code> pragmatic MVP choice (offsets).
LangSmith (optional)	Observability/tracing platform for LangChain/LangGraph apps — logs every LLM call, prompt, and node transition	Would give full visibility into every step of the pipeline for debugging/cost-tracking	<code>app/config.py</code> (<code>langchain_tracing_v2</code> , <code>langchain_project</code> , <code>langchain_api_key</code>)	Config flags exist and a <code>vars</code> but tracing is off by default ("false") — it's wired up optional/toggleable, not
Model Context Protocol (MCP)	An open protocol (client/server) for exposing "tools" (callable functions with typed schemas) to AI applications in a standardized way	Decouples FinePrint's core logic from <i>how</i> Slack/Drive/Gmail/Okta APIs work — new integrations = new MCP server, zero changes to agent code	<code>app/mcp/*</code> , standalone servers: <code>mock_drive_mcp.py</code> , <code>mock_slack_mcp.py</code> , <code>sqlite_mcp.py</code>	<code>McpClientManager.get_client</code> either a <code>MockMcpSession</code> (network) or a <code>RealMcpSession</code> subprocess speaking MCP using the official <code>mcp</code> Python expose the same <code>.call_with_params</code> interface.
pgvector-backed embeddings	Numeric vector representations of text, used to find "similar" text via distance metrics	Powers a lightweight RAG pattern: "have we seen a similar situation before, and did it work out?"	<code>agents/decision.py</code> (<code>_generate_embedding</code>), <code>models/decision.py</code> (<code>embedding</code> column)	Currently implemented as a mock/placeholder : a SQL text is expanded into a <code>SQL</code> (<code>_generate_embedding</code>) — commented "Replace with real embeddings". The plumbing (storage, search, prompt injection) only the embedding <i>mock</i> stubbed for the MVP.
JWT (python-jose)	Signed, stateless authentication tokens	Lets the FastAPI backend authenticate requests without server-side session storage	<code>app/auth/jwt.py</code>	<code>create_access_token(sub=org_id, role=role, expires_at=expires_at)</code> <code>get_current_user()</code> is a FastAPI dependency that decodes <code>Authorization: Bearer token</code> on protected routes.
RBAC (custom)	Role-Based Access Control	Ensures only <code>admin</code> / <code>user</code> roles can upload contracts	<code>app/auth/rbac.py</code>	<code>require_role([...])</code> is a factory; pre-built guards <code>require_user_or_admin</code> , <code>require_admin</code>

Technology	What it is	Why FinePrint uses it	Where it's used	How it works here
		or approve decisions; viewer role is read-only		are imported directly into signatures.
cryptography (Fernet-style encryption)	Symmetric encryption library	MCP server credentials (OAuth tokens, API keys) must never sit in the DB as plaintext	<code>app/mcp/crypto.py</code> , <code>mcp_connections.auth_credentials_encrypted</code>	Credentials are encrypted and decrypted only in-memory before establishing an MCP connection.
Redis	In-memory data store, commonly used for caching/queues	Configured (<code>redis_url</code> in settings, <code>rq/apscheduler</code> in requirements) for future job-queue use	<code>app/config.py</code>	Present in the architecture/dependencies but the MCP actually uses plain Python <code>threading.Thread + time.sleep()</code> for background jobs (see §14).
APScheduler / rq	Python job-scheduling / task-queue libraries	Intended production mechanism for periodic scans and background jobs	<code>requirements.txt</code>	Listed as a dependency running code (<code>main.py</code> , <code>verifier.py</code>) uses simple threads with <code>time.sleep()</code> rather than fully wiring up <code>rq/APScheduler</code> — a known simplification (see §14).
pypdf2, python-docx, pytesseract, Pillow	Document-parsing libraries (PDF text extraction, Word doc parsing, OCR for scanned images)	Contracts arrive as PDF/DOCX and sometimes scanned images — need to convert to plain text before any LLM can read them	<code>app/ingestion/document_parser.py</code>	<code>extract_text_from_bytes()</code> file extension; raises a <code>ParseFailure</code> exception (files, which the API layer <code>status = parse_failed</code> so the dashboard instead of disappearing).
fastapi-mail	Async email-sending library for FastAPI	Sends real notification emails (high-risk alerts, approval requests, send confirmations)	<code>app/services/email_service.py</code>	Fired in background thread so email sending never blocks HTTP response.
Next.js 14+ (App Router) / React 19	Full-stack React framework	Modern file-based routing (<code>app/contracts/[id]/page.tsx</code>), server + client components, fast dev experience	<code>frontend/src/app/*</code>	Pages: dashboard (<code>page</code> <code>contracts/</code> , <code>contracts/[id]</code> route for detail view), <code>api</code> <code>actions/</code> , <code>activity/</code> , <code>settings/</code>
Tailwind CSS v4	Utility-first CSS framework	Rapid, consistent styling without hand-writing CSS files; supports the custom light/dark "premium GitHub-style" UI	<code>frontend/src/app/globals.css</code> , all components	Class-based styling directives (<code>className="..."</code>).
TypeScript	Typed superset of JavaScript	Compile-time safety for the API client and component props	Entire <code>frontend/src</code>	<code>lib/api.ts</code> defines a type wrapper around <code>fetch</code> , <code>verify</code> per backend endpoint.
lucide-react	Icon component library	Consistent, lightweight SVG icons across the dashboard	Various components (<code>Sidebar.tsx</code> , <code>StatusBadge.tsx</code> , etc.)	Imported as React components (<code><AlertTriangle /></code>).
Docker Compose	Multi-container local orchestration	Spins up Postgres + Redis + backend + frontend together for local development	<code>docker-compose.yml</code>	Single command (<code>docker compose up</code>) launches the full stack — "modular monolith, single VM/container" deployment the HLD doc.
pytest / pytest-asyncio	Testing frameworks	Backend test suite (<code>test_pipeline.py</code>) exercises	<code>backend/test_pipeline.py</code>	Standard Python testing as both an HTTP client and server.

Technology	What it is	Why FinePrint uses it	Where it's used	How it works here
asyncio		the LangGraph pipeline logic		FastAPI's recommender transport.

6. AI Deep Dive — LangGraph, LangChain, RAG, Agents, Embeddings

This section goes deep specifically on the AI/agentic pieces, since that's usually the part interviewers probe hardest.

6.1 What is an "agent" here, really?

In FinePrint, an "agent" is **not** an autonomous, tool-calling, multi-step ReAct-style agent that decides its own next action. It's simpler and more predictable: **each agent is a single Python function that (a) receives the current pipeline state, (b) optionally calls an LLM once with a tightly-constrained prompt demanding raw JSON, and (c) returns an updated state.** This is an important nuance:

FinePrint uses the *word* "agent" in the product/business sense ("Detection Agent," "Risk Agent") but implements each one as a **deterministic single-shot LLM call wrapped in a graph node** — not a looping, tool-calling autonomous agent. The *orchestration* (which agent runs next, and under what condition) is handled by LangGraph's graph structure, not by the LLM deciding for itself.

This is a deliberate and important design choice: predictable, auditable, cheap, and impossible for the AI to "go off script" — a huge plus for a compliance/legal-adjacent product.

6.2 LangGraph — the orchestration engine

What it is: LangGraph models a workflow as a **graph of nodes and edges** operating over a shared, typed state object. It's built for exactly the kind of "multi-step, conditional, occasionally-parallel, must-be-resumable" pipelines that FinePrint needs.

Why FinePrint needs it instead of just calling functions in sequence in Python:

- **Conditional routing** is declarative (`add_conditional_edges`) instead of nested if/else spaghetti.
- **Checkpointing:** LangGraph can persist state between node executions (here, to Postgres via `langgraph-checkpoint-postgres`), so a contract re-scan can resume from where it left off instead of re-running (and re-paying for) every LLM call.
- **State merging semantics:** the `mcp_tool_calls` field in `ContractScanState` is declared as `Annotated[list[dict], operator.add]`, which tells LangGraph *"if two nodes try to update this key at the same time, concatenate the lists instead of raising a conflict error."* This is a LangGraph-specific mechanism for safe parallel writes.
- **Visual/mental clarity:** the graph *is* the documentation of the pipeline.

How it's built in FinePrint (`orchestrator/graph.py`):

```
graph = StateGraph(ContractScanState)
graph.add_node("detect_clauses", run_detection_agent)
graph.add_node("manual_review", manual_review_node)
graph.add_node("fetch_usage_signals", fetch_usage_signals_node)
graph.add_node("risk_agent", run_risk_agent)
graph.add_node("finance_agent", run_finance_agent)
graph.add_node("decision_agent", run_decision_agent)
graph.add_node("rule_check", apply_approval_rules)
graph.add_node("notify", notify_node)
graph.add_node("auto_log", auto_log_node)

graph.set_entry_point("detect_clauses")
graph.add_conditional_edges("detect_clauses", route_after_detection,
    {"manual_review": "manual_review", "continue": "fetch_usage_signals"})
graph.add_edge("manual_review", END)
graph.add_edge("fetch_usage_signals", "risk_agent")
graph.add_edge("risk_agent", "finance_agent")
graph.add_edge("finance_agent", "decision_agent")
graph.add_edge("decision_agent", "rule_check")
graph.add_conditional_edges("rule_check", route_after_rules,
    {"awaiting_approval": "notify", "auto_log": "auto_log"})
graph.add_edge("notify", END)
graph.add_edge("auto_log", END)

fineprint_graph = graph.compile()
```

Note the code comment: *"Sequential pipeline (no parallel fan-out) to avoid INVALID_CONCURRENT_GRAPH_UPDATE."* Interestingly, the HLD design documents originally called for Risk and Finance to run **in parallel** (a "fan-out/fan-in" pattern), and the state schema (`Annotated[list,`

operator.add]) was clearly built to support that. The final implementation runs them **sequentially** instead — a real-world example of “the design doc described the ideal, the shipped code took the simpler, safer path.” (Great interview talking point — see §14.)

6.3 The Shared State Object

```
class ContractScanState(TypedDict):
    contract_id: str
    org_id: str
    raw_text: str
    file_name: str
    clauses: Optional[dict]
    detection_confidence: Optional[float]
    usage_signals: Optional[dict]
    risk_output: Optional[dict]
    finance_output: Optional[dict]
    decision_output: Optional[dict]
    requires_approval: bool
    mcp_tool_calls: Annotated[list[dict], operator.add]
    error: Optional[str]
    route: str
```

Think of this as a **clipboard that gets passed down an assembly line** — every station reads what’s already written on it and adds its own notes, without erasing what came before. By the time it reaches the end, the clipboard contains the complete story of the contract’s analysis: extracted facts, usage data, risk classification, dollar figures, the final recommendation, and every external system call made along the way.

6.4 The five nodes in detail

Node	LLM call?	Temperature	Purpose	Key safety mechanism
Detection	 Yes	0.0	Extract vendor, renewal date, auto-renew flag, notice period, price escalation %, annual value	If confidence < 0.6 (configurable), routes to manual_review instead of guessing
Risk	 Yes	0.0	Classify risk type (underused / price_escalation / short_notice_window / none) and severity	Prompt explicitly says “Do NOT infer or fabricate usage numbers” — if no usage data exists, it’s told to say so, not make it up
Finance	 No	—	Compute exact dollar exposure	Zero LLM calls, zero MCP calls — pure Python float math. This is the load-bearing safety wall of the whole system.
Decision	 Yes	0.1	Write the human-readable narrative + recommend an action	LLM’s own savings_annual number is overwritten by the Finance Agent’s deterministic figure after the LLM responds — the LLM is only trusted for wording, never for the number
Action (post-approval only)	 Yes	0.3	Draft the outbound email/Slack text	Only runs <i>after</i> a human clicks Approve; even then, drafts are stored with status="draft" and require a second, explicit "Send" click

6.5 Robust JSON parsing from the LLM

Because Groq can serve reasoning-style open models (e.g., Qwen) that sometimes wrap their answer in <think>...</think> blocks or markdown code fences, every agent uses the same defensive helper:

```
def _extract_json(raw: str) -> str:
    raw = re.sub(r"<think>.*?</think>", "", raw or "", flags=re.DOTALL) # strip reasoning traces
    raw = re.sub(r"```(?:json)?", "", raw).replace("```", "").strip() # strip markdown fences
    match = re.search(r"\{.*\}", raw, re.DOTALL) # grab the first {...} blob
    return match.group(0) if match else raw
```

This is a very practical, real-world pattern: **never fully trust an LLM to follow output-format instructions perfectly** — always defensively parse.

6.6 RAG (Retrieval-Augmented Generation) — how it's implemented here

Classic RAG = "before asking the LLM to answer, retrieve relevant documents/facts from a knowledge base and stuff them into the prompt."
FinePrint implements a **narrow, purpose-built version of this** inside the Decision Agent:

1. Build a short text summary of the current situation: `f"Vendor: {vendor_name} - Risk: {risk_type}"`.
2. Convert it to a vector embedding (`_generate_embedding`).
3. Query Postgres/pgvector for the **3 closest past** Decision **rows that have a linked** Outcome **marked** success — using `Decision.embedding.l2_distance(embedding)` (Euclidean distance in vector space, ascending = most similar first).
4. Format them as plain text: `"- Past action: cancel, Result: Verified Complete..."`.
5. Inject that text into the Decision Agent's prompt under `PAST SIMILAR OUTCOMES: .`

Why this matters: the LLM isn't just reasoning in a vacuum — it's shown *"here's what happened last time a similar vendor/risk combination came up, and it worked."* This nudges the recommendation toward strategies that have actually proven successful for this organization before — a feedback loop from **Outcome** → **future Decision quality**.

Important honesty note (great interview material): the embedding function itself is currently a **placeholder** —

```
def _generate_embedding(text: str) -> list[float]:
    """Mock 1536-dimensional embedding based on text hash. Replace with LLM API call."""
    h = hashlib.sha256(text.encode()).digest()
    vec = [(b / 255.0) for b in h] * 48
    return vec
```

It hashes the text and repeats the 32 resulting bytes 48 times to fill a 1536-length vector (matching OpenAI's `text-embedding-3-small` dimensionality for schema-compatibility). This means **semantically similar text does not currently produce numerically similar vectors** — the retrieval is not yet "real" semantic search. But the *entire surrounding system* — the schema (`Vector(1536)` column), the migration that added it, the SQL similarity query, and the prompt-injection logic — is fully built and correct. Swapping in a real embedding model (e.g., an OpenAI/Cohere/local sentence-transformer call) is a one-function change. This is a textbook example of **shipping the scaffolding before the final model integration** — a completely normal and honest way to build an MVP.

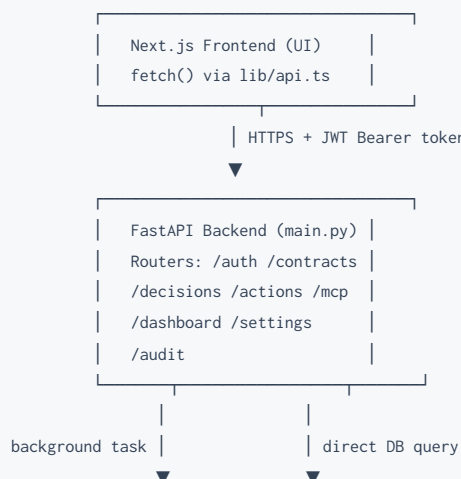
6.7 Why LangChain is present but lightly used

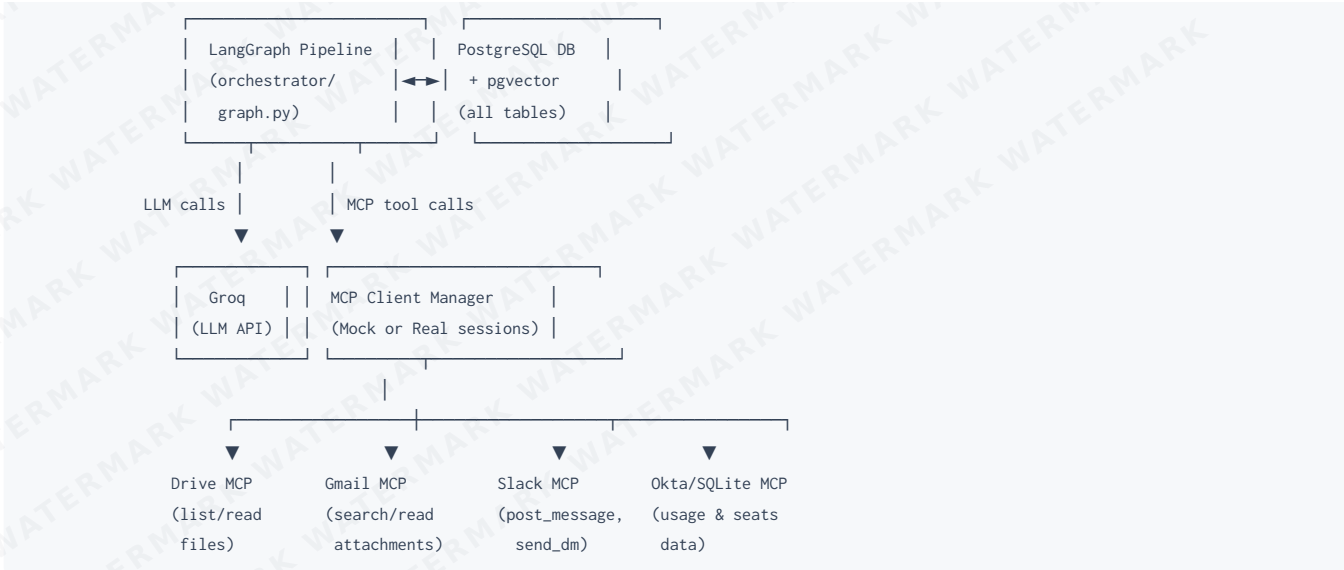
`langchain` and `langchain-groq` are listed as dependencies (LangGraph is technically part of the broader LangChain ecosystem and often installed alongside it), but the current agent code calls the **raw groq SDK directly** (`from groq import Groq; groq_client.chat.completions.create(...)`) rather than LangChain's `ChatGroq` wrapper or `PromptTemplate` abstractions. This is a legitimate, common pragmatic choice: for simple "one prompt in, one JSON out" calls, going straight to the provider SDK is less abstraction overhead than routing through LangChain's chat-model interface. LangGraph itself doesn't require LangChain chat models — a plain Python function is all a graph node needs to be.

6.8 Where an actual "vector database" concept lives

FinePrint doesn't use a dedicated vector database product (Pinecone, Weaviate, Chroma, etc.) — it uses **pgvector**, a Postgres extension that adds vector storage + similarity search directly inside the relational database it's already using. This is a legitimate, increasingly common architectural choice: **one database instead of two**, at the cost of not having a purpose-built vector index's most advanced features (e.g., HNSW at very large scale) — perfectly fine at FinePrint's scale (one embedding per decision, not millions of document chunks).

7. How Everything Connects





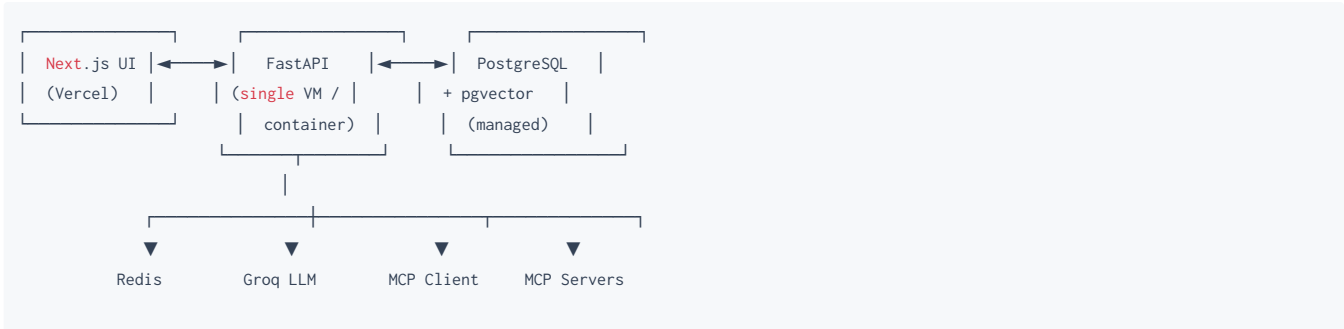
The golden rule of the connections: the LangGraph pipeline and the MCP layer never talk to Slack/Drive/Gmail/Okta *directly* – they always go through the `McpClientManager`, which decides (per org, per server type) whether to hand back a `MockMcpSession` (stub data, `MOCK_MCP=true`, the default) or a `RealMcpSession` (spawns an actual MCP server subprocess and speaks the protocol over stdio). This means the *entire pipeline can run and be demoed with zero real credentials* – a very deliberate MVP-friendly design.

8. Architecture Diagrams

8.1 System Context Diagram



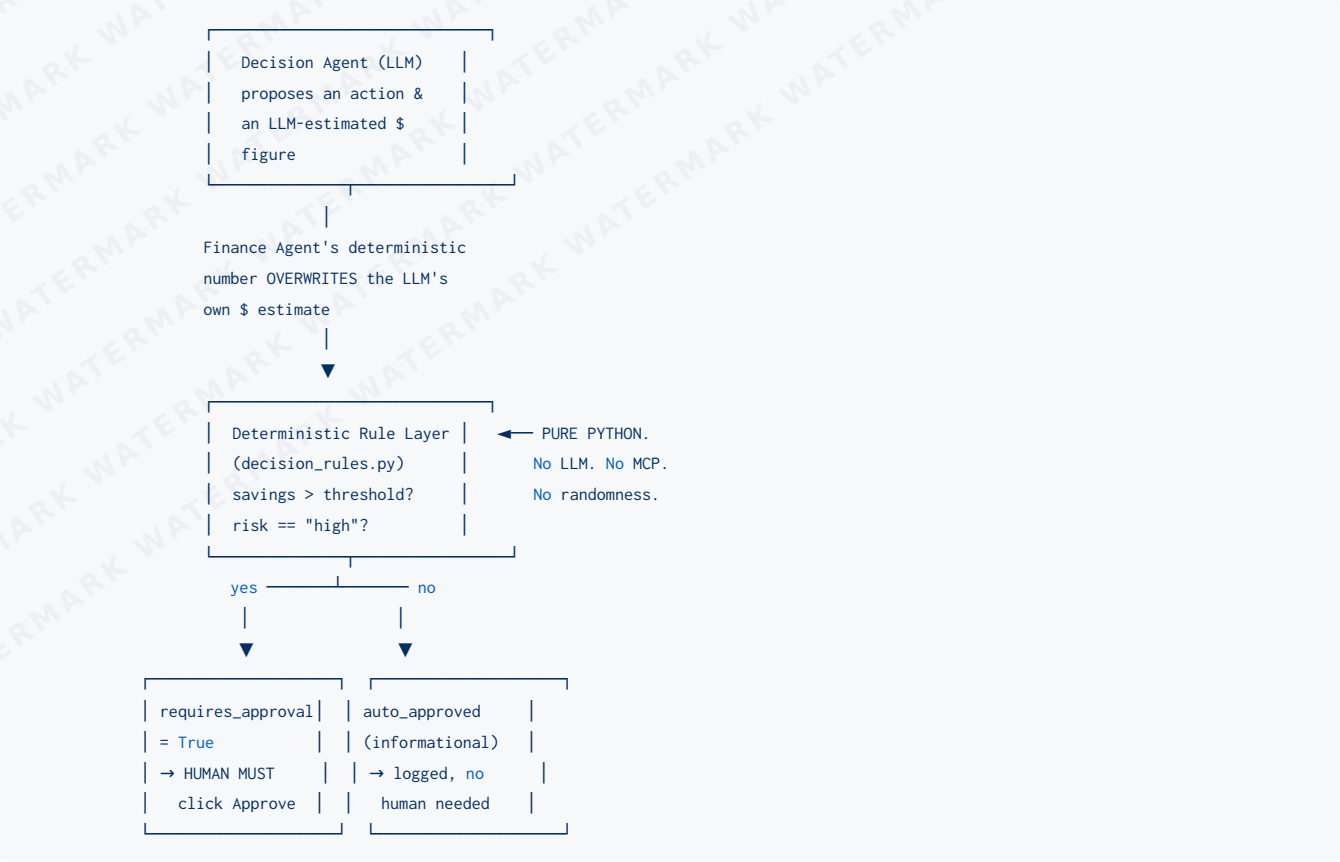
8.2 Deployment View



(job queue, agent state)	API	Layer	(Drive/Gmail/ Slack/Okta)
-----------------------------	-----	-------	------------------------------

Deliberately simple: no Kubernetes, no Kafka, no multi-region setup — "modular monolith" for MVP scale.

8.3 The Approval Safety Gate (the most important diagram in the whole system)



8.4 Entity Relationship (Data Model) Diagram

```
organizations 1—* users
organizations 1—* contracts 1—* contract_clauses
contracts      1—* agent_runs
contracts      1—* decisions 1—* actions
decisions      1—1 outcomes
organizations 1—* mcp_connections
organizations 1—1 org_settings
organizations 1—* audit_logs
organizations 1—* usage_signals
```

9. Complete Data Flow: Input → Output

Step-by-step, exactly what data looks like at each hop:

```
1. INPUT
  User uploads "high_risk_marketing_vendor.pdf"
  → POST /contracts/upload (multipart/form-data)

2. PARSING
  document_parser.extract_text_from_bytes(bytes, filename)
  → raw_text: "MASTER SERVICES AGREEMENT... Client shall pay Agency
    a fixed retainer of $12,500 USD per month... automatically
    renew for successive 12-month periods... 30% early termination
    penalty..."
```

3. DEDUPLICATION
deduplication.compute_hash(file_bytes) → SHA-256 string
check_duplicate() queries Contract.file_hash → not a duplicate → proceed

4. PERSIST
Contract row created: {status: "active", raw_text: "...", org_id: ...}

5. TRIGGER SCAN
POST /contracts/{id}/scan → BackgroundTasks.add_task(_run_pipeline, ...)

6. LANGGRAPH INVOKE
fineprint_graph.invoke({
 "contract_id": "...", "org_id": "...", "raw_text": "...",
 "clauses": None, ..., "route": "continue"
})

7. DETECTION NODE OUTPUT
{
 "vendor_name": "Global Marketing Dynamics LLC", "renewal_date": null,
 "auto_renew": true, "notice_period_days": 90, "price_escalation_pct": null,
 "contract_value_annual": 150000, "currency": "USD", "confidence": 0.9,
 "ambiguous_clauses": ["30% early termination penalty clause"]
}

8. USAGE SIGNALS (MCP call to Okta/SQLite mock)
{
 "active_users": 2, "total_seats": 15
} (or vendor not found → skip)

9. RISK NODE OUTPUT
{
 "risk_type": "short_notice_window", "risk_severity": "high",
 "evidence": "Auto-renews with only 90 days notice and a punitive
 30% early termination penalty clause"
}

10. FINANCE NODE OUTPUT (pure math, no LLM)
{
 "annual_cost_current": 150000.0, "annual_cost_if_renewed": 150000.0,
 "estimated_savings_if_cancelled": 150000.0,
 "estimated_savings_if_renegotiated": 37500.0, "currency": "USD"
}

11. DECISION NODE OUTPUT (LLM narrative + RAG context + overwritten \$ figure)
{
 "situation": "Contract contains predatory early termination penalties",
 "root_cause": "30% penalty clause discourages any exit before term end",
 "recommended_action": "renegotiate_seats",
 "expected_impact": {"savings_annual": 37500.0, "description": "..."},
 "risk": "high", "confidence": 0.88
}

12. RULE CHECK (deterministic)
org threshold = \$5,000. 37,500 > 5,000 → requires_approval = True
→ route = "awaiting_approval"

13. PERSISTENCE
ContractClause row + Decision row (approval_status="pending") saved.
Email + Slack (MCP) notifications fired on background threads.

14. HUMAN APPROVAL
Manager opens Approvals page → sees AI Analysis vs Policy Evaluation
side-by-side → clicks Approve
→ POST /decisions/{id}/approve

15. ACTION DRAFT (LLM, post-approval only)
Action row created: {
 "subject": "Re: Contract Review – Global Marketing
 Dynamics", "body": "...professional renegotiation email...",
 "recipient_hint": "Global Marketing Dynamics Account Manager"
}
status = "draft"

16. HUMAN SEND
User clicks Send on the Actions page → POST /actions/{id}/send
→ MCP tool call: slack.post_message(channel="#legal", text="...")

→ Action.status = "sent"

17. OUTPUT

A drafted, human-approved Slack message is posted to the Legal channel;
a full audit trail (extraction → risk → finance → decision → rule check
→ approval → draft → send) is stored and viewable per contract.

18. VERIFICATION (30 days later, background job)

Verifier re-queries usage via MCP. If active_users == 0 for the vendor,
Outcome.result = "success", spend_delta_amount = \$37,500 recorded.
This Outcome later feeds RAG context for future similar decisions.

10. Walkthrough Example (Realistic, Step-by-Step)

This mirrors the project's own README example almost exactly (it's a great one to memorize for a demo/interview), using the actual sample contract shipped in `test_docs/high_risk_marketing_vendor.txt`:

"Client shall immediately pay Agency the full remaining balance of the contract term plus a 30% early termination penalty... automatically renew for successive 12-month periods... written notice of non-renewal no later than ninety (90) days prior..."

Step	What Happens
Ingest	User uploads the file via the Dashboard. <code>AuditLog</code> records <code>Contract uploaded</code> .
Detect	LLM extracts: vendor = <i>Global Marketing Dynamics LLC</i> , annual value ≈ \$150,000 (12 × \$12,500), auto_renew = true, notice_period_days = 90.
Risk	Flags Clause 4 (30% early termination penalty + only 90-day opt-out window) → <code>risk_severity: high</code> .
Finance	Pure Python computes potential renegotiation savings (e.g., 25% of annual value for high-severity risk = \$37,500) and cancellation savings (full annual cost).
Decision	LLM writes: <i>"Contract contains predatory early termination penalties"</i> → recommends <code>renegotiate_seats</code> .
Rule Check	Org's <code>approval_threshold_usd = \$5,000</code> . <code>\$37,500 > \$5,000</code> → <code>requires_approval = True</code> .
Human-in-the-loop	Alert appears on the Approvals page. UI splits AI Analysis (LLM output) from Policy Evaluation (deterministic threshold check) so the manager sees exactly <i>why</i> it was flagged — not just <i>that</i> it was flagged.
Approve	Manager clicks Approve.
Draft	Action Agent generates a Slack message: <i>"High risk contract detected for Global Marketing Dynamics. Please review Clause 4 regarding a 30% termination penalty before signing."</i>
Send	Manager clicks Send. FinePrint calls the Slack MCP server — the core backend never needed to know how the Slack API actually works.
Verify (30 days later)	Background job checks vendor usage again to confirm whether the renegotiation/cancellation actually took effect.

11. File-by-File / Function-by-Function Reference

Orchestration core

File	Key symbols	Role
<code>orchestrator/state.py</code>	<code>ContractScanState</code> , <code>MCPToolCallLog</code>	Defines the shared TypedDict passed through every graph node; <code>MCPToolCallLog</code> standardizes how tool calls are logged for audit

File	Key symbols	Role
<code>orchestrator/graph.py</code>	<code>build_graph()</code> , <code>fineprint_graph</code> , <code>manual_review_node</code> , <code>fetch_usage_signals_node</code> , <code>auto_log_node</code> , <code>notify_node</code> , <code>route_after_detection()</code> , <code>route_after_rules()</code>	Wires all nodes/edges into the compiled LangGraph pipeline; <code>fineprint_graph</code> is the importable singleton used by the API layer

Agents

File	Key symbols	Role
<code>agents/detection.py</code>	<code>run_detection_agent()</code> , <code>_extract_json()</code>	First LLM call: raw text → structured clause JSON; routes to manual review below confidence threshold
<code>agents/risk.py</code>	<code>run_risk_agent()</code>	Second LLM call: classifies risk type/severity from clauses + usage data
<code>agents/finance.py</code>	<code>run_finance_agent()</code> , <code>_safe_float()</code> , <code>RENEGOTIATION_SAVINGS_FACTOR</code>	Pure-Python dollar math — no LLM, ever
<code>agents/decision.py</code>	<code>run_decision_agent()</code> , <code>_generate_embedding()</code> , <code>_extract_json()</code>	Synthesizes final recommendation; performs the RAG-style historical lookup; overwrites LLM \$ estimate with Finance Agent's number
<code>agents/action.py</code>	<code>run_action_agent()</code>	Post-approval-only LLM call that drafts the outbound email/Slack text

Rules & Governance

File	Key symbols	Role
<code>rules/decision_rules.py</code>	<code>apply_approval_rules()</code>	The deterministic gatekeeper — reads org-specific <code>approval_threshold_usd</code> / <code>second_approver_threshold_usd</code> and decides <code>requires_approval</code>

MCP layer

File	Key symbols	Role
<code>mcp/client_manager.py</code>	<code>McpClientManager</code> , <code>MockMcpSession</code> , <code>RealMcpSession</code> , <code>mcp_client_manager</code>	Per-org session registry; abstracts "talk to a real MCP server" vs "return canned stub data"
<code>mcp/crypto.py</code>	<code>decrypt_credentials()</code> (+ <code>encrypt</code> counterpart)	Symmetric encryption for stored MCP credentials
<code>mcp/{drive,gmail,slack,okta}_tools.py</code>	Tool wrapper helpers	Thin per-connector helper modules

API routers

File	Key endpoints	Role
<code>api/auth.py</code>	<code>POST /auth/register</code> , <code>POST /auth/login</code> , <code>GET /auth/me</code>	Signup/login, JWT issuance
<code>api/contracts.py</code>	<code>GET /contracts</code> , <code>GET /contracts/{id}</code> , <code>POST /contracts/upload</code> , <code>POST /contracts/{id}/scan</code>	Contract CRUD + pipeline trigger + background execution (<code>_run_pipeline</code> , <code>_persist_pipeline_results</code>)
<code>api/decisions.py</code>	<code>GET /decisions</code> , <code>POST /decisions/{id}/approve</code> , <code>POST /decisions/{id}/reject</code>	Approval queue; approval also triggers <code>_generate_action_draft()</code>
<code>api/actions.py</code>	Draft listing, <code>POST /actions/{id}/send</code>	Human-confirmed send → MCP execution
<code>api/dashboard.py</code>	<code>GET /dashboard/summary</code>	Aggregate stats (total exposure, savings realized, pending count)
<code>api/mcp.py</code>	Connect/disconnect/list MCP connections	Manages <code>mcp_connections</code> rows

File	Key endpoints	Role
api/settings.py	Org policy thresholds	Reads/writes <code>org_settings</code>
api/audit.py	Audit log listing	Read-only view of <code>audit_logs</code>

Models (ORM) – see also §12

`organization.py`, `user.py`, `contract.py`, `contract_clause.py`, `agent_run.py`, `decision.py`, `action.py`, `outcome.py`, `usage_signal.py`, `audit_log.py`, `org_settings.py`, `mcp_connection.py`, `mixins.py` (shared `UUIDPrimaryKeyMixin`, `TimestampMixin`).

Ingestion & Verification

File	Role
ingestion/document_parser.py	Bytes → text, per file type; raises <code>ParseFailure</code>
ingestion/deduplication.py	SHA-256 hashing + duplicate lookup
ingestion/ingestion_service.py	<code>poll_drive_mcp()</code> , <code>poll_gmail_mcp()</code> — MCP-based document discovery
ingestion/scheduler.py	Background thread loop calling the pollers every cycle
verification/verifier.py	<code>run_daily_verification()</code> — the 30-day outcome-check job
jobs/retention.py	Data retention/cleanup

Frontend

Area	Files	Role
Pages	<code>app/page.tsx</code> (dashboard), <code>app/contracts/page.tsx</code> , <code>app/contracts/[id]/page.tsx</code> , <code>app/approvals/page.tsx</code> , <code>app/actions/page.tsx</code> , <code>app/activity/page.tsx</code> , <code>app/settings/page.tsx</code> , <code>app/login/page.tsx</code>	Next.js App Router pages, one per major dashboard section
Components	<code>RiskBadge</code> , <code>RiskMeter</code> , <code>AgentTrail</code> , <code>PipelineNode</code> , <code>StatCard</code> , <code>TimelineEvent</code> , <code>PolicyRuleRow</code> , <code>CommandPalette</code> , <code>Sidebar</code> , <code>AppShell</code> , <code>ThemeProvider</code> , <code>StatusBadge</code> , <code>CurrencyValue</code> , <code>EmptyState</code>	Reusable UI building blocks — e.g. <code>AgentTrail</code> / <code>PipelineNode</code> visualize the LangGraph pipeline stages in the UI; <code>RiskBadge</code> / <code>RiskMeter</code> render risk levels
API client	<code>lib/api.ts</code>	Typed wrapper around every backend endpoint, with JWT auto-attached and 401 → redirect-to-login handling

12. Database Schema Explained

```
organizations
└─ id, name, created_at

users
└─ id, org_id → organizations, email, role(admin/user/viewer), created_at

mcp_connections
└─ id, org_id, mcp_server_type(google_drive/gmail/slack/okta),
  mcp_server_url, auth_credentials_encrypted, scopes_granted,
  status(active/expired/disconnected), last_verified_at

contracts
└─ id, org_id, source(drive/gmail/manual_upload), file_name, file_hash,
  raw_text, status(active/archived/parse_failed/manual_review/scanning),
```

```

last_scanned_at

contract_clauses (Detection Agent's structured output)
└ id, contract_id → contracts, vendor_name, renewal_date, auto_renew,
  notice_period_days, price_escalation_pct, contract_value_annual,
  currency, extraction_confidence, raw_extraction_json, ambiguous_clauses

usage_signals
└ id, org_id, vendor_name, active_users_count, seats_purchased, source

agent_runs (full audit trail per agent execution)
└ id, contract_id, agent_name, input_json, output_json,
  reasoning_summary, confidence, mcp_tool_calls_json, status

decisions
└ id, contract_id, situation, root_cause, recommended_action
  (cancel/renegotiate_seats/renew/manual_review),
  expected_impact_json, risk_level(low/medium/high), confidence,
  embedding (Vector(1536), pgvector), requires_approval,
  requires_second_approver, approval_status
  (pending/approved/rejected/auto_approved), approved_by_user_id

actions
└ id, decision_id → decisions, action_type(email_draft/slack_alert/
  task_created/cancellation_email_sent/renegotiation_email_sent),
  mcp_server_used, payload_json, status(draft/sent/cancelled),
  executed_at, executed_by_user_id

outcomes (1:1 with decisions)
└ id, decision_id → decisions (unique), expected_outcome, actual_outcome,
  verified_at, result(success/failure/inconclusive), spend_delta_amount

audit_logs
└ id, org_id, user_id, action, entity_type, entity_id, timestamp

org_settings
└ id, org_id, approval_threshold_usd, second_approver_threshold_usd, ...

```

Why this schema design is worth noting in an interview:

- Contract → ContractClause → Decision → Action → Outcome is a clean, linear audit chain — you can always answer *"what did we extract, what did we decide, what did we do, and did it work?"* for any contract.
- agent_runs is a separate, generic audit table (not tied 1:1 to any specific agent's schema) so every agent type — even future ones — can log there without a migration.
- Outcome is 1:1 with Decision (unique=True on decision_id) — a decision is verified exactly once, keeping the feedback loop clean and unambiguous.
- embedding lives directly on Decision, not in a separate vector table — since every decision needs at most one embedding, this avoids an unnecessary join.

13. FAANG-Level Interview Preparation

13.1 The elevator pitch (30 seconds)

"FinePrint is a contract-risk-monitoring platform that combines LLM-based document understanding with a deterministic policy engine. An LLM pipeline, orchestrated with LangGraph, extracts structured facts from contracts and proposes risk assessments and recommendations, but all financial math and all approval-threshold logic run in plain, auditable Python — never inside a prompt. Anything above a configurable dollar threshold requires explicit human approval before any external action (email, Slack message) is sent, and every step — including every external tool call — is logged for audit. External integrations go through the Model Context Protocol, so adding a new connector never touches core agent logic."

13.2 Likely questions & strong answers

Q: Why use LangGraph instead of just calling a sequence of Python functions? A: You *could* — for this exact linear pipeline, a plain sequence of function calls would technically produce the same result. The value LangGraph adds is (1) **declarative conditional routing** instead of nested if/else, (2) **built-in checkpointing** so a re-scan of an unchanged contract can resume/skip completed nodes rather than re-paying for LLM calls, (3) a **standard state-merging contract** (`operator.add` for concurrent-safe fields) if/when the pipeline does need to fan out in parallel, and (4) the pipeline structure becomes self-documenting and easy to extend (adding a node is additive, not a refactor).

Q: Why does the Finance Agent never call an LLM? A: Because LLMs are probabilistic and can produce plausible-looking but wrong arithmetic — completely unacceptable when a decision recommending real money movement is on the line. By keeping 100% of the dollar math in deterministic Python, the system guarantees the numbers are *always* exactly reproducible and auditable, and the LLM is even prevented from "helpfully" second-guessing them — the Decision Agent's code explicitly **overwrites** whatever `savings_annual` figure the LLM produced with the Finance Agent's real number.

Q: What happens if the LLM returns malformed JSON? A: Every agent wraps its parsing in a defensive `_extract_json()` helper that strips `<think>` reasoning blocks and markdown fences and regex-extracts the first `{...}` blob, then a `try/except json.JSONDecodeError`. On failure, the agent doesn't crash the pipeline — it returns a safe fallback state (e.g., routes to `manual_review`, or returns a `confidence: 0.0` decision object) so a bad LLM response degrades gracefully into "ask a human" rather than silently producing garbage or crashing the request.

Q: How do you prevent the AI from taking a harmful/costly action autonomously? A: Three independent layers: (1) the **deterministic rule engine** sets `requires_approval` based on dollar thresholds and risk level — this logic is not influenced by the LLM at all; (2) even *approved* decisions only produce a **draft** — the Action Agent never sends anything, it only writes to the database with `status="draft"`; (3) the actual send requires a **separate explicit human click** (`POST /actions/{id}/send`), which is a second, independent authorization gate beyond the initial decision approval.

Q: How does multi-tenancy work? A: Every table that holds tenant data carries an `org_id` foreign key, and every query in the API layer filters by `current_user.org_id` (extracted from the validated JWT). MCP sessions are also cached per `(org_id, server_type)` tuple in the client manager's registry, so one org's Slack/Drive session can never be reused for another org.

Q: What's the RAG component, and is it "real" RAG? A: It's a narrow, purpose-built RAG pattern: before the Decision Agent writes its recommendation, the system does a pgvector similarity search over past `Decision` rows joined to `successful Outcome` rows, and injects the top 3 matches into the prompt as historical grounding. Architecturally it's fully real (vector column, similarity query, prompt injection) — the one honest caveat is that the embedding function is currently a deterministic hash-based placeholder rather than a real embedding model call, which is clearly commented in the code as a to-do. It's a good example of building the full data pipeline first and swapping in the "real" model behind a stable interface later.

Q: Why MCP instead of writing direct Slack/Drive SDK integrations? A: Direct SDK integrations mean every new connector (QuickBooks, DocuSign, HubSpot...) requires new OAuth flows, new client code, and new agent-side logic to call it. MCP standardizes "tool access" behind one interface (`call_tool(name, params)`), so the agent code and the ingestion/action logic never need to change when a connector is added or swapped — only a new MCP server needs to be stood up and connected. It also creates a natural, enforced permission boundary: an MCP connection is only ever granted the specific tool scopes it needs (e.g., Slack gets `post_message / send_dm` only, never channel-admin tools).

Q: What are the actual trade-offs/limitations of this design, if you're being honest? A: (Also see §14 in full.) A few real ones: the embedding function is a placeholder, not a real model; the HLD's parallel Risk+Finance fan-out was simplified to sequential execution in the shipped code; Redis/rq/APScheduler are in `requirements.txt` but the actual background jobs use simple Python threads with `time.sleep()` loops rather than a real job queue — fine for an MVP/single-instance deployment, but wouldn't horizontally scale as-is; and `MOCK_MCP=true` by default means the demo doesn't exercise real external API failure modes (rate limits, auth expiry) unless explicitly switched to real connections.

Q: How would you scale this to production? A: Replace the thread-based background loops with a real job queue (`rq/Celery`) backed by the already-present Redis, so scans and verification jobs can run across multiple worker processes/machines instead of one in-process thread per instance. Swap the placeholder embedding function for a real embedding API call. Split out the LangGraph pipeline execution into its own worker service if LLM-call volume grows enough to need independent scaling from the API tier (the HLD explicitly calls this out as the one legitimate reason to break the modular monolith apart). Add LangSmith tracing (the config hooks already exist) for observability into LLM cost/latency/failure rates per node.

Q: Why store both `contract_value_annual` on the clause AND recompute it in the Finance Agent rather than trusting the LLM's own dollar figures throughout? A: The Detection Agent's extracted `contract_value_annual` is a *read* of a fact stated in the contract text (a legitimate LLM extraction task — reading comprehension). The Finance Agent's outputs are *derived calculations* (percentages, multiplications) from that fact — a different, purely computational task that should never be delegated to a language model, however good its arithmetic usually is. FinePrint draws this exact line deliberately: **LLMs read; Python computes**.

Q: Walk me through what happens if two different contracts trigger conflicting recommendations for the same vendor. A: Not explicitly handled in the current schema (each `Decision` is scoped to one `Contract`), which is a fair thing to flag as a known limitation/extension point — a natural next step would be vendor-level aggregation (a `vendor_id` normalized across contracts) so recommendations can be reconciled at the vendor level rather than the contract level.

13.3 System-design whiteboard prompts you could get, based on this project

1. "Design a system that reviews legal documents and flags risk — how do you keep an LLM from taking unsupervised action on high-stakes decisions?" → Answer using the three-layer safety model (rule engine → draft-only → explicit send).
2. "How would you add a new external integration (e.g., DocuSign) to this system?" → Explain MCP: stand up a DocuSign MCP server exposing scoped tools, register a new `McpServerType` enum value, no agent code changes needed.
3. "How would you reduce LLM cost as contract volume grows 100x?" → LangGraph checkpointing to skip re-running unchanged nodes on re-scans; caching extracted clauses; capping `max_contracts_per_scan`; batching/rate-limiting.
4. "How do you make an AI pipeline auditable for compliance purposes?" → `agent_runs` table logging input/output/confidence/reasoning per node + `mcp_tool_calls_json` + immutable `audit_logs` table + full Decision→Action→Outcome chain.

14. Design Decisions & Trade-offs

Decision	What was chosen	Alternative considered	Trade-off
Monolith vs. microservices	Single FastAPI modular monolith	Microservices per domain (ingestion, agents, actions)	Simpler to build/deploy/debug at MVP scale; sacrifices independent scaling of hot paths (e.g., LLM-heavy scanning) until actually needed
Sequential vs. parallel agent execution	Risk → Finance run sequentially in the shipped graph	HLD/state design (<code>operator.add</code> on <code>mcp_tool_calls</code>) clearly anticipated parallel fan-out	Simpler, avoids <code>INVALID_CONCURRENT_GRAPH_UPDATE</code> state-merge conflicts; costs a small amount of latency (two sequential LLM/compute steps instead of overlapping them)
LLM does math vs. Python does math	Finance Agent is 100% deterministic Python	Let the LLM estimate savings directly	Removes hallucination risk entirely for the single highest-stakes number in the system, at the cost of the Finance model being only as sophisticated as its hard-coded heuristics (e.g., a fixed 10/15/25% renegotiation-savings-by-severity table)
Real embeddings vs. placeholder	Deterministic hash-based mock vector	Call a real embedding API (OpenAI, Cohere, local model)	Zero cost/dependency for MVP demoing; the RAG retrieval is not yet semantically meaningful until swapped — but the interface is model-agnostic and ready
Mock MCP vs. real MCP by default	<code>MOCK_MCP=true</code> default	Require real OAuth connections for every demo	Makes the whole system runnable/demoable instantly with zero external credentials; real integration behavior (auth expiry, rate limits) isn't exercised until explicitly flipped on
Threads vs. real job queue	<code>threading.Thread</code> + <code>time.sleep()</code> loops for schedulers/notifications	Redis + <code>rq</code> /APScheduler (already a dependency)	Fast to implement, works great single-instance; doesn't horizontally scale across multiple backend processes/machines without further work
Raw Groq SDK vs. LangChain chat model wrapper	Direct <code>groq</code> SDK calls	<code>langchain_groq.ChatGroq</code>	Fewer abstraction layers for simple single-shot JSON-extraction prompts; loses some LangChain-native features (e.g., built-in retry/fallback chains, standardized tool-calling) that would come "for free" through the LangChain interface
pgvector vs. dedicated vector DB	pgvector extension inside the existing Postgres	Pinecone / Weaviate / Chroma	One database to operate instead of two; less specialized at very large scale, but fine at "one embedding per decision" scale
Draft-then-send vs. auto-send on approval	Two separate human actions: Approve (decision) then Send (action)	Auto-send immediately after approval	Extra safety checkpoint — a human can review the actual drafted wording before it goes out, catching cases where the recommendation was right but the wording needs editing

15. Quick-Reference Cheat Sheet

One-liners for the whole stack:

- **Frontend:** Next.js 14+ / React 19 / TypeScript / Tailwind CSS — the dashboard UI.
- **Backend:** FastAPI (Python) — REST API + background task orchestration.
- **ORM:** SQLAlchemy 2.0 (typed `Mapped[...]` style) + Alembic migrations.
- **Database:** PostgreSQL + `pgvector` extension for embeddings.
- **LLM Provider:** Groq (fast inference; open-weight models like Qwen).
- **Agent Orchestration:** LangGraph `StateGraph` — 5 nodes, conditional routing, checkpointable.
- **Agent Framework Ecosystem:** LangChain / langchain-groq (present, lightly used directly).
- **Observability (optional):** LangSmith tracing hooks (off by default).
- **RAG:** `pgvector` similarity search over past successful `Decision / Outcome` pairs.
- **External Integrations:** Model Context Protocol (MCP) — Drive, Gmail, Slack, Okta, mockable.
- **Auth:** JWT (python-jose) + custom RBAC (admin/user/viewer).
- **Security:** Encrypted-at-rest MCP credentials, org-scoped multi-tenancy on every query.
- **Background jobs:** Python threads (MVP) for ingestion polling + 30-day outcome verification.
- **Core safety principle:** *AI suggests → deterministic rules govern → human approves → human sends.*

The 5 pipeline nodes, memorized:

1. **Detection** (LLM) — reads the contract, extracts facts.
2. **Risk** (LLM) — classifies the danger.
3. **Finance** (Python only) — computes the exact dollars.
4. **Decision** (LLM + RAG) — writes the recommendation, dollar figure is overwritten by Finance.
5. **Rule Check** (Python only) — decides if a human must approve.

The 5 core database entities, memorized: `Contract → ContractClause → Decision → Action → Outcome`

End of analysis. This document is designed to be read top-to-bottom for a full understanding, or used as a quick-jump reference (via the Table of Contents) before an interview or a team walkthrough.